

# LỘ TRÌNH THỰC CHIẾN 14 NGÀY

Xây Dựng Toàn Diện Môi Trường Kiểm Tra Tự Động Cho Parameterized FIFO RTL

Phân bổ chiến thuật: 25% RTL Refactoring & Kiến trúc | 75% Class-based Verification (SVA & Scoreboard)

**Mục tiêu cốt lõi:** Chuyển đổi từ tư duy kiểm tra thủ công bằng mắt (directed testbench + waveform) sang xây dựng môi trường kiểm tra tự động (Self-checking & Random Verification Environment) chuẩn công nghiệp. Hoàn thiện project nặng ký làm điểm nhấn cốt lõi trong CV thực tập hè năm 3.

## PHẦN 1: TƯ DUY THIẾT KẾ & PHÂN BỐ CÔNG SỨC

Trong môi trường thiết kế vi mạch chuyên nghiệp, công sức dành cho việc kiểm tra thiết kế (Verification) luôn chiếm từ 70% đến 80% tổng thời gian dự án. Một thiết kế RTL phức tạp đến đâu cũng trở nên vô giá trị nếu không thể chứng minh nó chạy đúng 100% trong mọi điều kiện biên (corner cases).

| THÀNH PHẦN          | TƯ DUY TIẾP CẬN CŨ (HỌC TẬP)                                                        | TƯ DUY THỰC CHIẾN (CÔNG NGHIỆP)                                                                         |
|---------------------|-------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------|
| RTL Design          | Hard-coded kích thước cố định (16x8), dùng hằng số trực tiếp trong logic.           | Tham số hóa hoàn toàn ( <b>parameter</b> ), tự động tính toán độ rộng con trỏ bằng hàm <b>\$clog2</b> . |
| Testbench Kiến trúc | Dạng tuyến tính, cấp tín hiệu thủ công trong khối <b>initial</b> .                  | Cấu trúc hướng đối tượng (Class-based), phân tách rõ ràng nhiệm vụ: Driver, Monitor, Scoreboard.        |
| Cơ chế Kiểm tra     | Nhìn mắt vào sóng tín hiệu (Waveform) hoặc dùng câu lệnh <b>\$display</b> đơn giản. | Sử dụng Reference Queue làm mô hình chuẩn, tự động so sánh dữ liệu đầu ra và bắt lỗi lập tức.           |
| Phát hiện Lỗi Biên  | Đoán mò tình huống rồi viết code test cố định cho trường hợp đó.                    | Rải SystemVerilog Assertions (SVA) bắt lỗi giao thức đồng thời chạy dữ liệu ngẫu nhiên (Randomization). |

## PHẦN 2: LỘ TRÌNH CHI TIẾT TỪNG NGÀY (HÀNH TRÌNH 14 NGÀY)

### Tuần 1: Chuẩn Hóa RTL & Dựng Khung Môi Trường Hướng Đối Tượng (OOP)

Tuần đầu tiên tập trung vào việc dọn dẹp mã nguồn RTL cũ và thiết lập nền móng kiến trúc vững chắc cho Testbench thông qua Interface và các Class cơ bản.

## Ngày 1: Tham Số Hóa Toàn Diện RTL (RTL Refactoring) RTL

- **Nhiệm vụ:** Loại bỏ các giá trị hard-code kích thước bộ nhớ 16 và độ rộng dữ liệu 8-bit. Thay thế bằng hai hằng số hệ thống `parameter DATA_WIDTH = 8` và `parameter DEPTH = 16`.
- **Yêu cầu kỹ thuật:** Sử dụng hàm toán học hệ thống `localparam ADDR_WIDTH = $clog2(DEPTH)` để tự động tính toán độ rộng cho con trỏ ghi (`wr_ptr`) và con trỏ đọc (`rd_ptr`).
- **Tối ưu:** Tạo hai tín hiệu nội bộ `wr_fire = wr_en && !full` và `rd_fire = rd_en && !empty` nhằm cô lập logic điều khiển chính, giúp code tường minh và dễ debug.

## Ngày 2: Đóng Gói Tín Hiệu Bằng SystemVerilog Interface Verification

- **Nhiệm vụ:** Khai báo một cấu trúc `interface fifo_if` gom toàn bộ các cổng kết nối của DUT (`clk`, `rst_n`, `wr_en`, `rd_en`, `din`, `dout`, `full`, `empty`).
- **Yêu cầu kỹ thuật:** Định nghĩa khối `clocking block` bên trong interface để đồng bộ hóa thời gian lấy mẫu dữ liệu đầu vào và đầu ra theo sườn dương của xung clock, loại bỏ triệt để hiện tượng chạy đua tín hiệu (race conditions) khi mô phỏng.
- **Kết nối:** Chuyển đổi file testbench top cũ sang cách gọi instance của Interface và map vào DUT thông qua cổng interface vừa tạo.

## Ngày 3: Định Nghĩa Transaction Class & Cơ Chế Cấp Phát Ngẫu Nhiên Verification

- **Nhiệm vụ:** Tạo dựng class `fifo_transaction` mô tả một đơn vị dữ liệu hoặc một chu kỳ thao tác truyền nhận qua FIFO.
- **Yêu cầu kỹ thuật:** Sử dụng từ khóa `rand` cho các thuộc tính điều khiển: `rand bit wr_en;`, `rand bit rd_en;`, và `rand bit [DATA_WIDTH-1:0] din;`.
- **Hiện thực:** Viết các phương thức hỗ trợ cơ bản như `function void display()` để in nhanh trạng thái gói tin phục vụ mục đích gỡ rối (debug) luồng dữ liệu sau này.

## Ngày 4: Hiện Thực Class Driver (Bộ Tạo Kích Thích Giao Tiếp) Verification

- **Nhiệm vụ:** Viết class `fifo_driver` đóng vai trò chuyển đổi các gói tin trừu tượng cấp cao từ Transaction thành chuỗi sự kiện nhị phân trên các đường dây vật lý.
- **Yêu cầu kỹ thuật:** Driver nhận các đối tượng dữ liệu, chờ sườn xung clock đồng bộ từ interface, sau đó trực tiếp gán giá trị của gói tin vào các ngõ vào `vif.cb.wr_en`, `vif.cb.rd_en`, và `vif.cb.din` thông qua liên kết con trỏ ảo `virtual interface`.

## Ngày 5: Hiện Thực Class Monitor (Bộ Quan Sát & Thu Thập Dữ Liệu) Verification

- **Nhiệm vụ:** Xây dựng class `fifo_monitor` chạy độc lập song song với Driver, có nhiệm vụ giám sát bị động toàn bộ hành vi trên bus của FIFO.
- **Yêu cầu kỹ thuật:** Cứ mỗi chu kỳ clock, Monitor liên tục đọc các giá trị thực tế đang diễn ra trên các chân tín hiệu của interface. Nếu phát hiện một chu kỳ đọc hoặc ghi thành công (*fire*), nó lập tức đóng gói toàn bộ trạng thái vào một đối tượng Transaction mới và đẩy về cho tầng xử lý cấp cao.

## Ngày 6 & 7: Thiết Kế Mô Hình Tự Động Kiểm Tra (Self-Checking Scoreboard) Verification

- **Nhiệm vụ:** Xây dựng trái tim của môi trường kiểm tra: Class `fifo_scoreboard`. Đây là thành phần giúp loại bỏ hoàn toàn việc phải mở sóng đồ thị lên xem bằng mắt.
- **Cơ chế hoạt động (Reference Model):** Khai báo một mảng động không giới hạn kích thước (SystemVerilog Native Queue) làm hàng đợi tham chiếu chuẩn: `logic [DATA_WIDTH-1:0] ref_queue[$];`.
- **Logic đối chiếu:**
  - Khi Monitor báo cáo có sự kiện ghi dữ liệu thành công: Hệ thống tự động đẩy giá trị `din` vào cuối hàng đợi bằng hàm `ref_queue.push_back()`.
  - Khi Monitor báo cáo có sự kiện đọc dữ liệu thành công: Hệ thống thực hiện lấy phần tử đầu tiên trong hàng đợi ra bằng hàm `ref_queue.pop_front()` và so sánh trực tiếp với dữ liệu ngõ ra thực tế `dout` thu được từ chip.
- **Báo cáo:** Tự động tăng biến đếm lỗi `mismatch_count` và xuất thông báo lỗi nghiêm trọng bằng câu lệnh hệ thống `$error` nếu dữ liệu không trùng khớp hoặc sai thứ tự truyền nhận.

## Tuần 2: Rủi Xác Suất Ngẫu Nhiên, Cài Đặt Khối Assertions & Vét Cạn Góc Khuất (Corner Cases)

Tuần thứ hai tập trung nâng cấp độ khó của bài kiểm tra bằng cách ép mạch hoạt động dưới áp lực dữ liệu ngẫu nhiên có ràng buộc và chứng minh độ tin cậy bằng Assertion.

### Ngày 8: Thiết Lập Ràng Buộc Kịch Bản (Constraint Randomization) Verification

- **Nhiệm vụ:** Tạo class `fifo_generator` quản lý việc tạo kịch bản kiểm tra và áp dụng các hàm ràng buộc ( `constraint` ) để điều hướng luồng dữ liệu ngẫu nhiên một cách thông minh.
- **Kỹ thuật viết ràng buộc:**
  - Định nghĩa kịch bản 1 (Giai đoạn đầu): Ràng buộc sao cho tỉ lệ `wr_en` xuất hiện đạt 90% nhằm nhanh chóng dồn ép dữ liệu đẩy FIFO đạt trạng thái chạm ngưỡng Full.
  - Định nghĩa kịch bản 2 (Giai đoạn hoạt động thông thường): Ràng buộc phân bố xác suất đồng đều 50% Ghi và 50% Đọc để kiểm tra khả năng xử lý liên tục của con trỏ bộ nhớ.

### Ngày 9: Kết Nối Toàn Diện Môi Trường Qua Class Environment Verification

- **Nhiệm vụ:** Khởi tạo class tổng `fifo_env` đứng ra làm nhạc trưởng kết nối các thành phần đơn lẻ lại với nhau thông qua cơ chế hòm thư chia sẻ dữ liệu ( `mailbox` ).
- **Yêu cầu kỹ thuật:** Sử dụng khối lệnh thực thi song song `fork ... join_none` để kích hoạt toàn bộ các tiến trình hoạt động độc lập của Generator, Driver, Monitor và Scoreboard cùng chạy đồng thời ngay khi bài test bắt đầu bắt nhịp xung clock.

### Ngày 10 & 11: Cài Đặt Rào Chặn Giám Sát Giao Thức Bằng SystemVerilog Assertions (SVA) Verification

- **Nhiệm vụ:** Khai báo các câu lệnh kiểm tra ràng buộc thuộc tính (Assertions) trực tiếp bên trong Interface hoặc nhúng vào RTL để theo dõi hành vi mạch theo thời gian thực từng chu kỳ.
- **Các kịch bản SVA bắt buộc phải triển khai:**
  1. **Kiểm tra trạng thái Reset:** `assert property (@(posedge clk) !rst_n |-> (empty == 1 && full == 0));` (Đảm bảo khi có reset tích cực, cờ trạng thái phải trả về giá trị mặc định ngay lập tức).
  2. **Bảo vệ biên khi Cạn (Underflow):** `assert property (@(posedge clk) (empty && !wr_en && rd_en) |-> => $stable(rd_ptr));` (Chứng minh rằng nếu đang empty mà cố tình đọc và không ghi, con trỏ đọc tuyệt đối không được phép dịch chuyển hay thay đổi vị trí bộ nhớ).
  3. **Bảo vệ biên khi Tràn (Overflow):** `assert property (@(posedge clk) (full && wr_en && !rd_en) |-> => $stable(wr_ptr));` (Đảm bảo bộ ghi bị khóa chặt khi bộ nhớ đã đầy).

## Ngày 12 & 13: Đột Phá Điểm Số Phỏng Vấn Với Việc Vết Sạch Corner Cases Verification

- **Nhiệm vụ:** Nhà tuyển dụng cấp cao luôn tập trung hỏi cách ứng viên xử lý các điểm mù của mạch số. Hãy chủ động lập trình ép mạch rơi vào các trạng thái nguy hiểm nhất.
- **Kịch bản Corner Cases thực chiến cần thiết lập:**
  - Đọc và ghi đồng thời (`wr_en=1, rd_en=1`) liên tục ngay tại thời điểm FIFO đang ở trạng thái mấp mé sát giới hạn Full (`DEPTH - 1`) hoặc mấp mé sát giới hạn Empty (`1`).
  - Tạo kịch bản dồn ép dữ liệu liên tục vượt qua giới hạn bộ nhớ để kiểm tra tính năng quay vòng của con trỏ địa chỉ (*Pointer Wrap-around logic*).
  - Kích hoạt xung hạ reset đột ngột bất đồng bộ tích cực ngay giữa thời điểm lưu lượng traffic truyền dữ liệu ngẫu nhiên đang chạy ở mức đỉnh điểm cao nhất để xem hệ thống có tự phục hồi an toàn về trạng thái tĩnh ổn định hay không.

## Ngày 14: Thu Thập Log Đạt PASS, Tổng Hợp Số Liệu Bằng Báo Cáo LaTeX Report

- **Nhiệm vụ:** Chuyển đổi thành quả kỹ thuật thành một sản phẩm portfolio chuyên nghiệp để trình bày trực quan trước hội đồng tuyển dụng.
- **Triển khai báo cáo chuyên nghiệp:** Sử dụng hệ thống tài liệu LaTeX để thiết lập một file báo cáo PDF kỹ thuật chuẩn. Vẽ lại sơ đồ cấu trúc khối phân tách môi trường kiểm tra của bạn. Trích xuất toàn bộ lịch sử log ghi nhận trạng thái từ Scoreboard kèm theo các minh chứng toàn bộ hệ thống câu lệnh kiểm tra SVA đều đạt trạng thái `0 errors / PASS` sau hàng chục ngàn chu kỳ mô phỏng ngẫu nhiên.

## PHẦN 3: QUY TẮC "SINH TỒN" TRONG PHÒNG THÍ NGHIỆM

Để đạt được kết quả chuyển biến rõ rệt nhất về mặt tư duy và kỹ năng thực chiến trong vòng 14 ngày, bạn bắt buộc phải tuân thủ nghiêm ngặt 3 nguyên tắc thép sau:

1. **Nguyên tắc "Tư duy độc lập trước khi gõ":** Tuyệt đối không copy-paste bất kỳ đoạn mã nguồn class testbench nào từ các tài liệu có sẵn trên mạng hay nhờ trí tuệ nhân tạo sinh sẵn toàn bộ cấu trúc. Hãy tự tay gõ từng dòng lệnh khai báo Class, từng khối lệnh điều khiển thời gian đồng bộ để ghi nhớ sâu sắc cơ chế hướng đối tượng.
2. **Nói lớn suy nghĩ khi Debug (Thinking Out Loud):** Khi môi trường testbench báo lỗi đỏ hoặc hiển thị sai lệch dữ liệu giữa Reference Model và DUT, hãy tập thói quen giải thích to thành tiếng suy nghĩ phân tích dòng dữ liệu của mình bằng ngôn ngữ kỹ thuật (Ví dụ: *"Hiện tại dữ liệu đầu ra bị chậm mất 1 chu kỳ so với hàng đợi mô phỏng, lỗi có thể nằm ở việc gán tín hiệu của Monitor chưa đồng bộ đúng clocking block..."*). Đây chính là cách rèn luyện phản xạ vàng giúp bạn đạt điểm tối đa trong vòng phỏng vấn kỹ thuật trực tiếp.
3. **Đóng băng mục tiêu cốt lõi:** Tuyệt đối tuân thủ phạm vi tài liệu định hướng đã chọn. Trong suốt quá trình 14 ngày này, không sa đà mở rộng mã nguồn sang các cấu trúc phức tạp như Asynchronous FIFO, đồng bộ đa miền dòng quét xung clock (CDC), mã hóa con trỏ sang hệ mã Gray, hay cố gắng tích hợp các thư viện công kênh như UVM. Việc sở hữu một bộ mã nguồn Synchronous FIFO đơn giản nhưng đi kèm môi trường tự động kiểm tra bao phủ tuyệt đối mọi góc khuất chức năng luôn có giá trị vượt trội trong mắt nhà tuyển dụng so với một mạch phức tạp nhưng testbench sơ sài, thiếu tính tự động.